

CC Assignment 01: 23K-2001 Date: _____

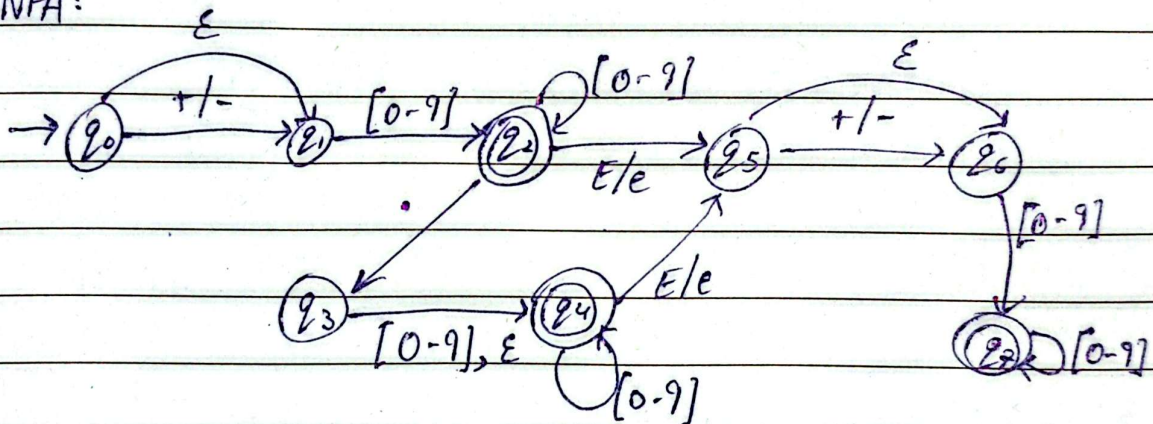
Q1. Regular expression:

$$[+-]?[0-9]^+(\.[0-9]^*)?([eE][+-]?[0-9]^+)?$$

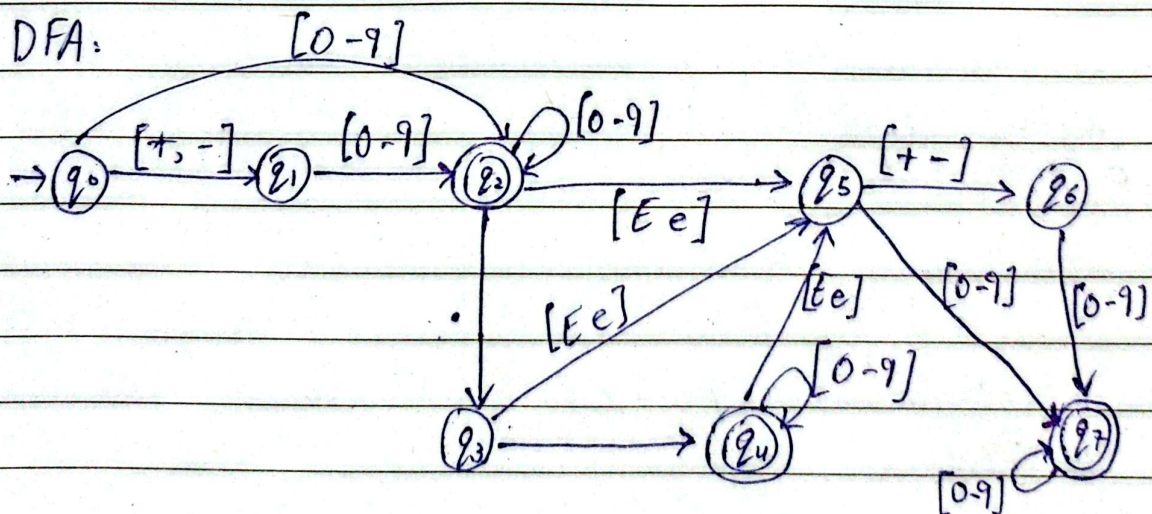
⇒ Breaking down:

- $[+-]^?$ → optional sign (+ or -)
- $[0-9]^+$ → followed by one or more digits
- $(\.[0-9]^*)$ → an optional decimal part
- $([eE][+-]?[0-9]^+)$ → an optional exponent part

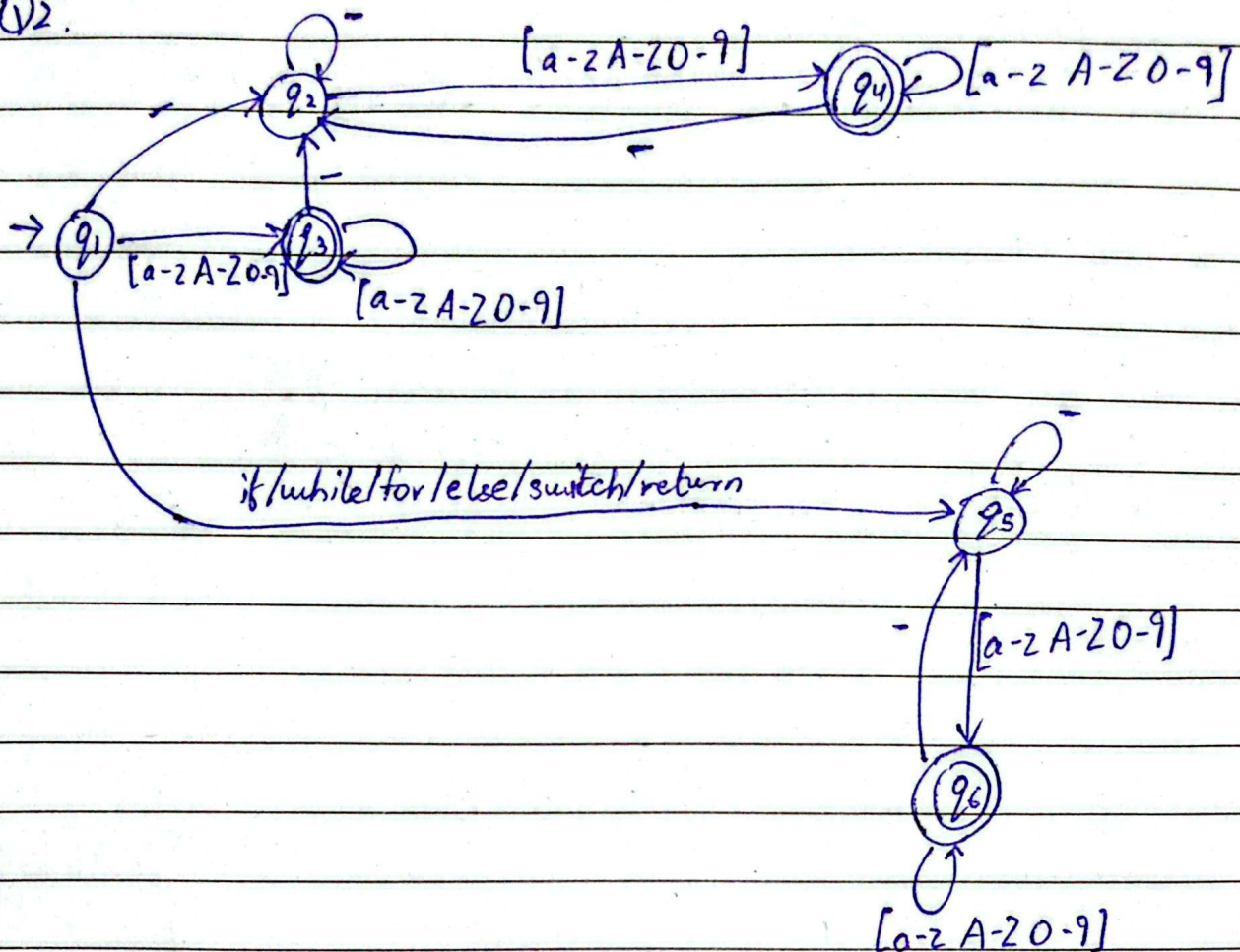
NFA:



DFA:



Q2.



Q3. Lexical Analyzer Rule:

A simple DFA can't handle nested comments as it requires counting. Hence, a counter is used to track nesting depth.

Rules:

On reading `"/*"`
level = 1

ENTER COMMENT STATE

IN COMMENT STATE

while level > 0

if `"/*"` encountered:

level += 1

else if `"*/"` encountered:
 `level -= 1`

else:

 consume next character
 return to INITIAL STATE

Finite Automaton:

A simple DFA can't handle nested comments because they have no memory, therefore we can use a counter to trick nesting logic, with the comment being fully processed when counter hits zero. This approach extends FA with memory similar to PDA.

STATES:

- S_0 → initial state
- S_1 → after reading `/`
- S_2 → inside comment
- S_3 → after reading `*` inside comments

TRANSITIONS:

- | | |
|--|--------------------------------------|
| • S_0 | • S_2 (inside comment) |
| ⇒ $S_0 \rightarrow S_1$ on <code>/</code> | On <code>/</code> : |
| ⇒ Otherwise stay in S_0 | look ahead |
| | if next is <code>*</code> : |
| • S_1 | stay in S_2 , <code>level++</code> |
| ⇒ $S_1 \rightarrow S_2$ on <code>*</code> , <code>level=1</code> | otherwise: |
| ⇒ Otherwise $S_1 \rightarrow S_0$ (no comment) | stay in S_2 |

• S_3 (after * inside comment)

On $/$:

→ level--

→ if level == 0

$S_3 \rightarrow S_0$ (comment ends)

On $*$:

→ $S_3 \rightarrow S_3$ (stay)

→ on any other characters

$S_3 \rightarrow S_2$

⇒ Final condition:

The automation exits the comment only when level = 0.

Q4. → this is a blankspace

Input string: int - count _ = - 100;

→ Assuming buffer size is 8.

Buffer 1: int - coun

Buffer 2: t _ = - 100;

1) lexeme: int

- lexeme begin = 'i'
- forward → moves $i \rightarrow n \rightarrow t$
- stops at blankspace
- Token = KEYWORD
- lexeme begin = forward (after recognition)

3) lexeme: =

- lexeme begin = '='
- forward → moves one step
- token = ASSIGN-OP
- lexeme begin = forward.

2) lexeme: count

- lexeme begin = 'c'
- forward → moves $c \rightarrow o \rightarrow u \rightarrow n \rightarrow t$ (buffer 1 ends)
- stop at blankspace
- token = Identifier
- lexeme begin = forward

4) lexeme: 100

- lexeme begin = '1'
- forward → moves $1 \rightarrow 0 \rightarrow 0$
- stops at ;
- token = NUMBER
- lexeme begin = forward

5) lexeme: ;

- lexeme begin = ';'
 - forward $\rightarrow$ moves one step
 - token = SEMICOLON

$\Rightarrow$ When forward reaches end of a buffer.

- The next buffer is combined with the next part of input.
- If the lexeme spans buffers (like "count"), forward continues without stopping.
- An EOF marker at the end of buffer signals that the lexer should reload the next buffer.

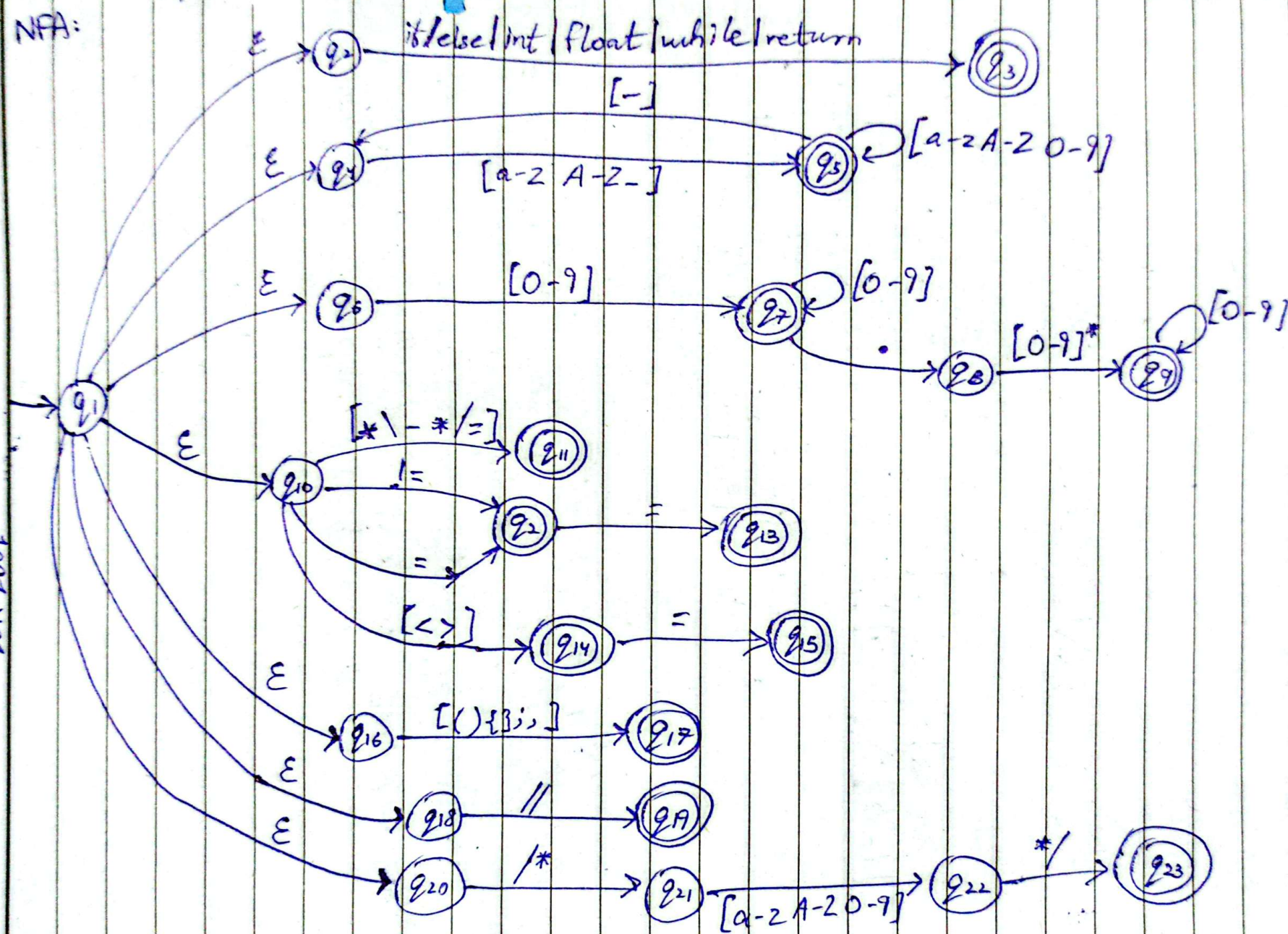
Q5. Regular expressions:

- keywords: `int | float | if | else | while | return`
- identifier: `[a-zA-Z_][a-zA-Z0-9_]*`
- integer constant: `[0-9]+`
- float constant: `[0-9]+ \. [0-9]* | [0-9]* \. [0-9]+`
- operators: `[+|-|*|/|=|<|>|<=|>=|<|>]`
- delimiter: `[(){}; ,]`
- single line comment: `// [^\n]*`
- multiline comment: `/\^+ [ \s \S ]* ? \* /`

NFA:

Date:

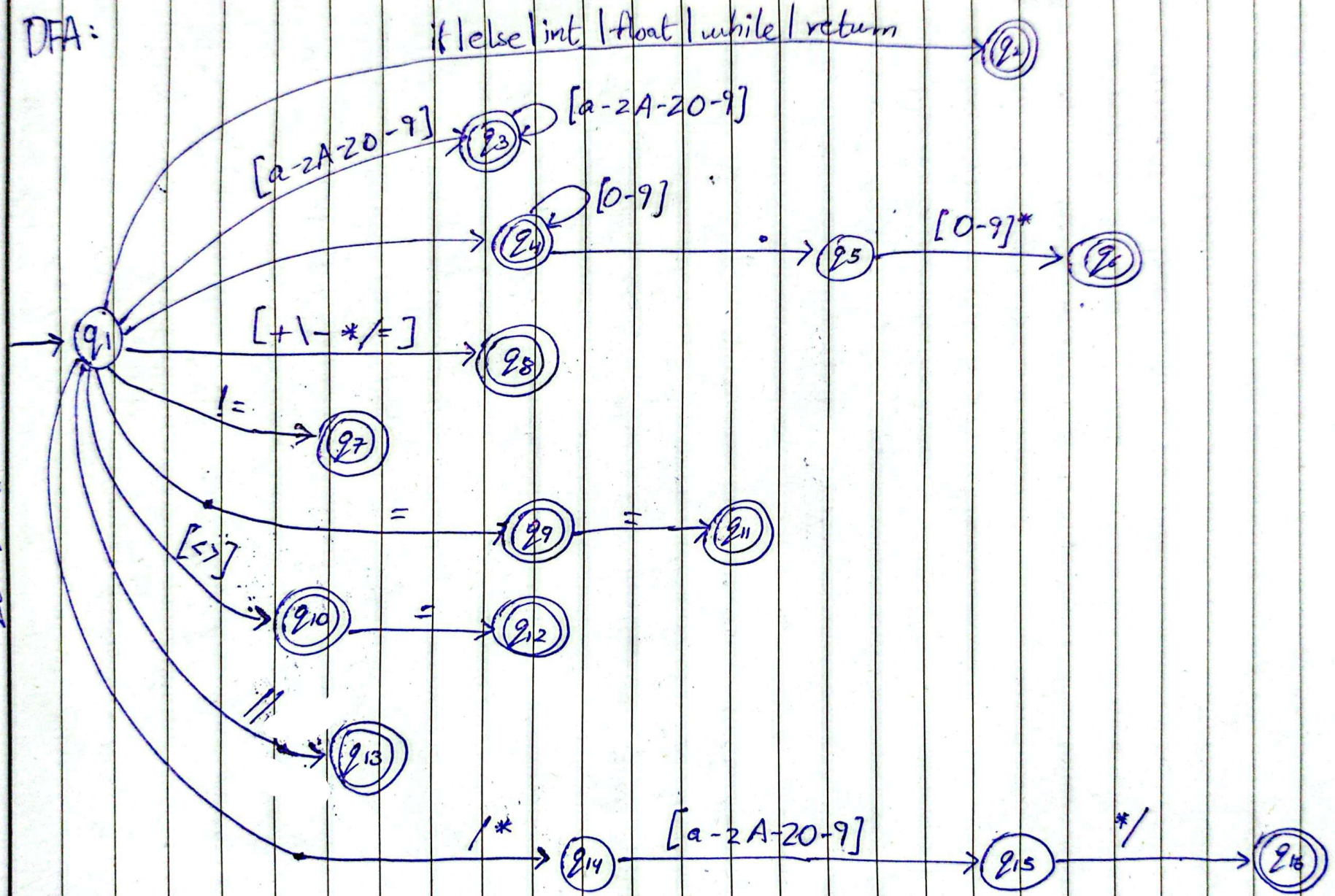
23K-2001



DFA:

Date:

23K-2001



Q6. CFG's belong to Type 2 in the Chomsky Hierarchy. CFG's are more powerful than regular expressions because they can represent recursive and nested structures while R.E cannot. R.E's are limited to regular language and have no memory, so they can't handle dependencies like matching number of symbols. In contrast CFG's can describe such patterns, making them more expressive. For eg:

$$L = \{a^n b^n \mid n \geq 0\}$$

CFG for this language: $S \rightarrow aSb \mid \epsilon$

⇒ This cannot be regular as it requires equal number of a's and b's and since FA's can't keep track of count as they have no memory, this cannot be regular.

Q7. CFG for simplified JSON:

$S \rightarrow \text{object}$

$\text{Object} \rightarrow \{\} \mid \{\text{members}\}$

$\text{members} \rightarrow \text{pair} \mid \text{pair}, \text{members}$

$\text{pair} \rightarrow \text{string} : \text{value}$

$\text{value} \rightarrow \text{string} \mid \text{number} \mid \text{object} \mid \text{array} \mid \text{true} \mid \text{false} \mid \text{null}$

$\text{array} \rightarrow [] \mid [\text{elements}]$

$\text{elements} \rightarrow \text{value} \mid \text{value}, \text{elements}$

23K-2001

Q1. CFG for scientific calculator:

$$S \rightarrow E$$

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * P \mid T / P \mid P$$

$$P \rightarrow F^P \mid F$$

$$F \rightarrow (E) \mid \text{NUMBER} \mid \text{CONSTANT} \mid \text{FUNC}(E)$$

$$\text{FUNC} \rightarrow \text{SIN} \mid \text{COS} \mid \text{TAN} \mid \ln \mid \text{LOG} \mid \text{sqrt}$$

$$\text{CONSTANT} \rightarrow \text{pi} \mid e$$